

Анализа графова транзитних мрежа

Паралелна имплементација BFS, WCC и PageRank алгоритама

Марко Кубурић

Индекс: E2 62/2025

Факултет техничких наука, Универзитет у Новом Саду

Фебруар 2026.

Сажетак

Овај рад представља имплементацију система за анализу графова транзитних мрежа у програмском језику Rust. Систем користи CSR репрезентацију и имплементира три алгоритама (BFS, WCC, PageRank) у секвенцијалној и паралелној варијанти. Експериментална евалуација на графовима са до милион чворова показује убрзање до $7,3\times$ за PageRank при 32 нити.

1 Увод

Анализа великих графова представља централни проблем у друштвеним мрежама, транспортним системима и веб претраживању. Овај пројекат имплементира систем `fast-transit-network-analytics` за анализу графова са три кључна алгоритама: **BFS** (претрага у ширину), **WCC** (слабо повезане компоненте) и **PageRank** (рангирање чворова).

За сваки алгоритам развијене су секвенцијална и паралелна верзија коришћењем `Rayon` библиотеке. Циљ рада је анализа скалабилности паралелних имплементација и идентификација уских грла.

2 Имплементација

2.1 CSR репрезентација графа

Compressed Sparse Row (CSR) је стандардна репрезентација ретких графова. Граф са n чворова и m грана се представља помоћу два низа:

- `offsets`: низ дужине $n + 1$ где `offsets[v]` означава почетак суседа чвора v ,
- `edges`: низ дужине m са одредишним чворовима свих грана.

Суседи чвора v се налазе на позицијама `[offsets[v], offsets[v + 1])`. За алгоритме који захтевају улазне гране (PageRank), имплементација подржава реверзну CSR структуру.

2.1.1 Зашто CSR?

CSR репрезентација је изабрана из неколико кључних разлога:

1. **Меморијска ефикасност:** CSR захтева само $O(n + m)$ меморије, што је оптимално за ретке графове. Алтернативе попут матрице суседства захтевају $O(n^2)$ меморије, што је неприхватљиво за графове са милионима чворова.

2. **Cache-friendly приступ:** Суседи сваког чвора су складиштени континуално у меморији, што омогућава секвенцијално читање и максимално искоришћење CPU кеша. Код обиласка графа, ово значајно убрзава приступ суседима у односу на листе повезаности где су суседи раштркани по меморији.
3. **Паралелизација:** CSR омогућава лаку поделу посла - сваки чвор има јасно дефинисан опсег суседа, па више нити може независно обрађивати различите чворове без потребе за закључавањем структуре графа.
4. **Константан приступ суседима:** Дохватање листе суседа чвора v је $O(1)$ операција (израчунавање индекса), док је код листе повезаности потребно $O(\deg(v))$ за алокацију и праћење показивача.

За бенчмарк графове, меморијска потрошња износи: мали граф ($n=262K$, $m=4M$) захтева ≈ 19 - 38 MB, велики граф ($n=1M$, $m=16M$) захтева ≈ 76 -- 151 MB (зависно од тога да ли се користи реверзна CSR структура за PageRank).

3 Алгоритми

3.1 BFS - претрага у ширину

BFS (Breadth-First Search) је алгоритам за обилазак графа који открива све чворове достижне из изворног чвора s , рачунајући најкраће растојање (у броју грана) до сваког од њих.

3.1.1 Секвенцијална имплементација

Алгоритам одржава FIFO ред чворова за обраду и низ растојања `dist[]` иницијализован на -1 (недостижан):

1. Постави `dist[s] = 0` и додај s у ред
2. Док ред није празан:
 - (a) Извади чвор u из реда
 - (b) За сваког суседа v од u : ако је `dist[v] = -1`, постави `dist[v] = dist[u] + 1` и додај v у ред

Временска сложеност је $O(n + m)$ јер се сваки чвор и грана обрађују тачно једном. За неусмерене графове (представљене као усмерени), скенирају се и излазне и улазне гране.

3.1.2 Паралелна имплементација

Паралелни BFS користи **level-synchronous** приступ где се у свакој итерацији паралелно обрађују сви чворови на истом растојању од извора:

1. **Frontier паралелизам:** Тренутни frontier (скуп чворова на растојању d) се дели међу нитима помоћу `par_iter()`. Свака нит независно обрађује подскуп frontier чворова.
2. **Атомске операције:** Низ растојања је имплементиран као `Vec<AtomicI64>`. Када нит покуша да посети чвор v , користи `compare_exchange(-1, d+1)` - ово атомски проверава да ли је v непосећен и ако јесте, поставља растојање. Само нит која успешно изврши CAS додаје v у следећи frontier.

3. **Локални бафери:** Свака нит акумулира новооткривене чворове у локални вектор, избегавајући контенцију на заједничкој структури. На крају итерације, локални бафери се спајају помоћу `reduce()` у нови `frontier`.
4. **Баријера синхронизације:** Између нивоа постоји имплицитна баријера - следећи ниво не почиње док се тренутни не заврши.

3.2 WCC - слабо повезане компоненте

WCC (Weakly Connected Components) проналази компоненте повезаности у графу третираном као неусмерени - два чвора су у истој компоненти ако постоји пут између њих (игноришући смер грана).

3.2.1 Секвенцијална имплементација

Алгоритам итеративно покреће BFS из непосећених чворова:

1. Иницијализуј низ `comp[]` на ∞ (непосећен)
2. За сваки чвор v где је `comp[v] =  $\infty$` :
 - (a) Покрени BFS из v , означавајући све достижне чворове са `comp[u] = v`
 - (b) BFS скенира и излазне и улазне суседе (третира граф као неусмерени)
3. Резултат: `comp[u] = comp[w]` ако и само ако су u и w у истој компоненти

Сложеност је $O(n + m)$ јер се сваки чвор и грана посете тачно једном.

3.2.2 Паралелна имплементација

Паралелизација WCC користи исти level-synchronous BFS као код паралелног BFS-а:

1. **Спољна петља:** Итерација по непосећеним чворовима остаје секвенцијална - када се пронађе непосећен чвор, покреће се паралелни BFS за ту компоненту.
2. **Атомски ID компоненте:** Низ компоненти користи `AtomicU32` са иницијалном вредношћу `u32::MAX`. CAS операција `compare_exchange(MAX, comp_id)` осигурава да сваки чвор буде означен тачно једном.
3. **Унутрашњи паралелизам:** За сваку компоненту, `frontier` се обрађује паралелно као код BFS-а. Веће компоненте имају више посла и боље искоришћавају паралелизам.

Ограничење: ако граф има много малих компоненти, већина времена се троши на секвенцијално покретање BFS-а за сваку.

3.3 PageRank

PageRank је итеративни алгоритам за рангирање важности чворова у графу, оригинално развијен за рангирање веб страница. Интуиција: чвор је важан ако на њега указују важни чворови.

3.3.1 Секвенцијална имплементација (pull варијанта)

Алгоритам итеративно ажурира рангове док не конвергирају:

1. Иницијализуј $r_v^{(0)} = 1/n$ за све чворове
2. У свакој итерацији t :
 - (a) Израчунај $S^{(t)} = \sum_{u: \deg^+(u)=0} r_u^{(t)}$ (сума рангова dangling чворова)
 - (b) За сваки чвор v , израчунај нови ранг:

$$r_v^{(t+1)} = \alpha \left(\sum_{u \in N^-(v)} \frac{r_u^{(t)}}{\deg^+(u)} + \frac{S^{(t)}}{n} \right) + \frac{1 - \alpha}{n} \quad (1)$$

где је $N^-(v)$ скуп улазних суседа, $\deg^+(u)$ излазни степен, и $\alpha = 0,85$

3. Понављај док L1 норма разлике $\|r^{(t+1)} - r^{(t)}\|_1 < \varepsilon$ или до max итерација

Pull варијанта значи да сваки чвор "повлачи" рангове од својих улазних суседа, што захтева реверзну CSR структуру.

3.3.2 Паралелна имплементација

PageRank има регуларну структуру која се добро паралелизује:

1. **Per-vertex паралелизам:** Израчунавање новог ранга за сваки чвор је независно од других чворова у истој итерацији. Користи се `into_par_iter()` преко свих чворова, при чему Rayon аутоматски дели посао међу нитима.
2. **Read-only приступ:** Све нити читају из истог низа старих рангова $r^{(t)}$ без модификације, па нема потребе за синхронизацијом приликом читања.
3. **Независни уписи:** Свака нит пише у различите позиције низа $r^{(t+1)}$, па нема race condition-a.
4. **Dangling sum:** Сума dangling чворова се рачуна секвенцијално пре паралелне фазе. Ово је потенцијално уско грло, али заузима мали део укупног времена.
5. **Баријера:** На крају сваке итерације, имплицитна баријера осигурава да су сви нови рангови израчунати пре провере конвергенције.

PageRank је **memory-bound** - брзина је ограничена пропусним опсегом меморије јер алгоритам чита рангове улазних суседа који су раштркани по меморији.

4 Генерисање тест графова

За бенчмаркинг је имплементиран генератор случајних графова по **Erdős–Rényi моделу** $G(n, m)$. Генератор креира тачно m грана бирајући униформно случајне парове чворова из скупа $[0, n)$.

4.0.1 Зашто Erdős–Rényi модел?

1. **Репродуцибилност:** Коришћењем фиксног seed-а за генератор псеудослучајних бројева, добијају се идентични графови при сваком покретању бенчмарка. Ово је кључно за поуздано поређење перформанси између различитих имплементација и конфигурација.
2. **Контролисана величина:** Модел $G(n, m)$ гарантује тачан број чворова и грана, што омогућава прецизну контролу величине улазних података и меморијске потрошње.
3. **Униформна дистрибуција степена:** За велике графове, степен чворова прати Поасонову дистрибуцију са средином $\bar{d} = 2m/n$. Ово даје балансирано оптерећење при паралелној обради - нема екстремно великих чворова који би постали уска грла.
4. **Једноставност имплементације:** Генерисање се своди на m позива генератора случајних бројева, без сложених структура података. Ово минимизира overhead генерисања и омогућава брзо креирање великих графова.
5. **Graph500 конвенција:** Параметризација прати стандард Graph500 бенчмарка:
 - $n = 2^{\text{SCALE}}$ чворова,
 - $m = \text{edgefactor} \times n$ грана (подразумевано edgefactor = 16).

Ово омогућава лако поређење са резултатима других система.

Генерисани бенчмарк графови:

- **Мали граф:** SCALE=18, $n = 262,144$ чворова, $m = 4,194,304$ грана,
- **Велики граф:** SCALE=20, $n = 1,048,576$ чворова, $m = 16,777,216$ грана.

Већи граф има $4\times$ више чворова и грана, што омогућава анализу скалабилности алгоритама у односу на величину улаза.

5 Верификација

Коректност се верификује на три нивоа:

1. **Јединични тестови:** на малим графовима са познатим резултатима,
2. **Поређење seq/par:** резултати секвенцијалне и паралелне верзије морају бити еквивалентни,
3. **CLI тестови:** интеграциони тестови комплетног pipeline-а.

Сви тестови (10 јединичних + 6 CLI + 3 compare) успешно пролазе.

6 Експериментална поставка

Хардвер: Intel Core i9-13900HX (24 језгра, 32 нити), 64 GB DDR5 RAM.

Софтвер: Fedora 43, Linux 6.17.12, rustc 1.92.0, release профил.

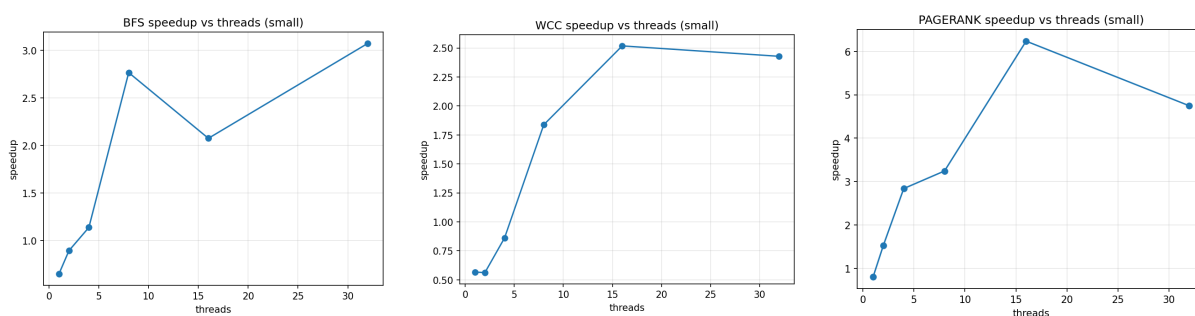
Убрзање се рачуна као $S(p) = T_{\text{seq}}/T_{\text{par}}(p)$.

7 Резултати

7.1 Мали граф

Табела 1: Убрзање за мали граф ($n = 262,144$)

Алгоритам	1	2	4	8	16	32
BFS	0,65	0,89	1,14	2,77	2,08	3,07
WCC	0,57	0,56	0,86	1,84	2,52	2,43
PageRank	0,81	1,53	2,84	3,25	6,24	4,75

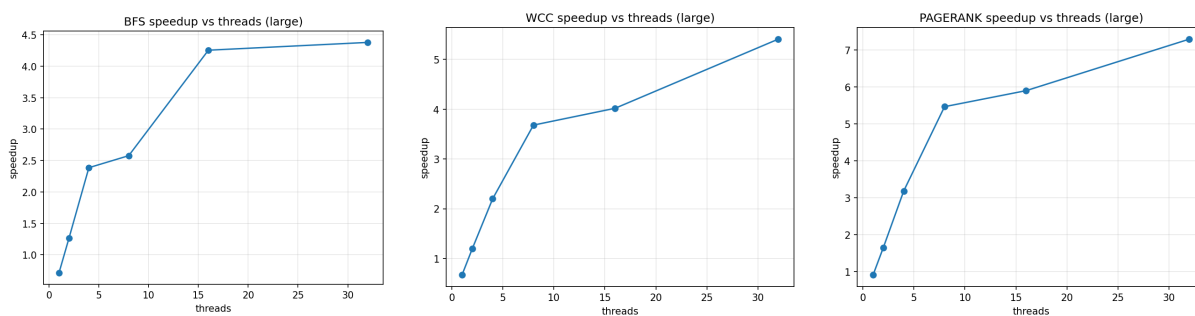


Слика 1: Убрзање у зависности од броја нити за мали граф

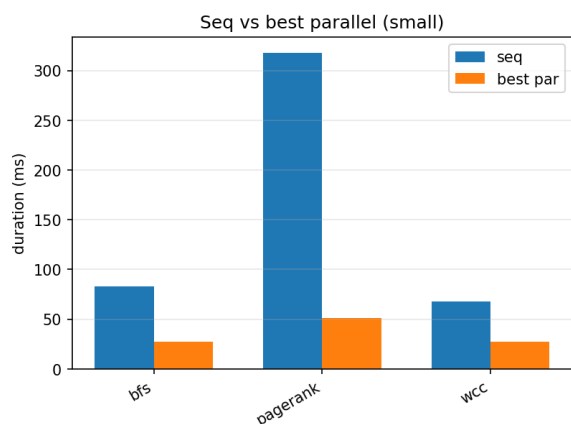
7.2 Велики граф

Табела 2: Убрзање за велики граф ($n = 1,048,576$)

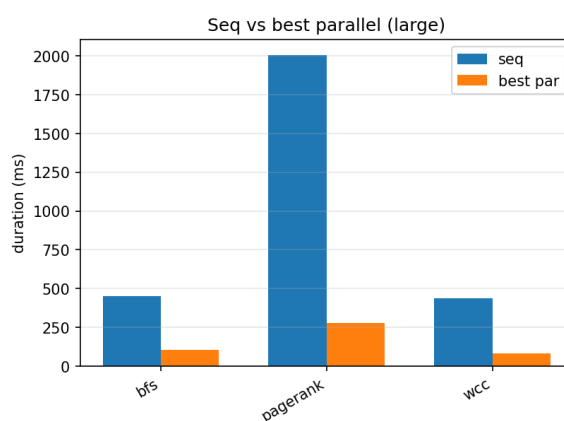
Алгоритам	1	2	4	8	16	32
BFS	0,71	1,26	2,39	2,58	4,26	4,38
WCC	0,67	1,20	2,20	3,68	4,02	5,41
PageRank	0,92	1,65	3,19	5,47	5,90	7,30



Слика 2: Убрзање у зависности од броја нити за велики граф



Слика 3: Seq vs par - мали граф



Слика 4: Seq vs par - велики граф

8 Дискусија

8.1 Overhead паралелизације

Сви паралелни алгоритми показују убрзање < 1 при једној нити (0,65--0,92), што указује на overhead атомских операција и thread pool управљања.

8.2 Скалабилност

Велики граф показује боље скалирање: BFS $4,38\times$ vs. $3,07\times$, WCC $5,41\times$ vs. $2,52\times$, PageRank $7,30\times$ vs. $6,24\times$. Већи графови боље амортизују фиксни overhead паралелизације.

8.3 Разлике између алгоритама

PageRank има најбоље скалирање због регуларне структуре - свака итерација обрађује све чворове равномерно. BFS и WCC имају ирегуларне обрасце приступа зависне од топологије, што доводи до неравномерног оптерећења и веће контенције.

8.4 Ефекат хипертрединга

Пад убрзања при 32 нити (нпр. PageRank мали: $6,24\times \rightarrow 4,75\times$) настаје због дељења ресурса између логичких језгара и засићења memory bandwidth-a.

9 Закључак

Главни закључци рада:

1. CSR репрезентација омогућава ефикасно складиштење са $O(n+m)$ меморије и cache-friendly приступ,
2. Паралелизација помоћу Rayon постиже убрзање до $7,3\times$ за PageRank,
3. Скалабилност зависи од величине графа - мали графови не амортизују overhead,
4. Регуларни алгоритми (PageRank) боље скалирају од ирегуларних (BFS, WCC).

Правци будућег рада: direction-optimizing BFS, паралелизација dangling sum рачунања, и оптимизација меморијског layout-a.